



School of Computer Science Engineering and Information Systems
Department of Information Technology
Winter 2025-26

NETWORK AND INFORMATION SECURITY
BITE401L

PROJECT REPORT

**“A Secure File Encryption and Real-Time
Sharing System with Digital Signatures and
HMAC Authentication”**

Submitted by

MATHIAZHAGAN G -23BIT0386

K SRIRAM-23BIT0393

T SAI DEVA HARSHA – 23BIT0331

Under the Guidance of

Dr. SHANTHARAJAH S P

Professor Higher Academic Grade, SCORE

VIT, Vellore.

1.Abstract

Modern file sharing systems often lack comprehensive security mechanisms, exposing sensitive data to risks such as unauthorized access, tampering, and impersonation. Traditional approaches typically rely on single-layer encryption techniques, which fail to provide complete guarantees of confidentiality, integrity, authentication, and non-repudiation. This project presents a **Secure File Encryption and Real-Time Sharing System with Digital Signatures and HMAC Authentication**, designed to ensure end-to-end security for file transmission in a web-based environment.

The system implements a multi-layered cryptographic framework combining **Fernet symmetric encryption (AES-128-CBC)** for file confidentiality, **RSA-2048 digital signatures** for authentication and non-repudiation, and **HMAC-SHA256** for integrity verification. The application is developed using a **Flask-based backend**, **SQLite database**, and a responsive **HTML/CSS/JavaScript frontend**, enabling secure real-time file sharing between users. Additional mechanisms such as secure key handling, user authentication, and tamper detection are integrated to strengthen overall system security.

Experimental evaluation demonstrates that the system achieves an encryption throughput of approximately **23.6 MB/s** with **100% tamper detection accuracy** under various test conditions. The application successfully passed **95 test cases** across functional, security, and performance categories with a **100% success rate**, indicating high reliability and robustness. These results confirm that the proposed system effectively provides secure, efficient, and real-time file sharing with strong cryptographic guarantees and minimal performance overhead.

2.Introduction

The security of file sharing systems depends on ensuring that data remains confidential, unaltered, and accessible only to authorized users during transmission and storage.

Traditional platforms often rely on basic encryption or access control mechanisms, which provide limited protection against modern threats. These approaches fail to guarantee key security properties such as integrity, authentication, and non-repudiation, making systems vulnerable to data breaches, tampering, and impersonation attacks.

With the growing use of web-based applications for real-time file sharing, risks such as data interception, unauthorized modification, and replay attacks have increased. In many systems, files may be encrypted but lack proper integrity checks or sender authentication, creating significant security gaps, especially when handling sensitive data.

This project addresses these challenges by developing a **Secure File Encryption and Real-Time Sharing System with Digital Signatures and HMAC Authentication**. The system combines symmetric encryption for confidentiality, digital signatures for authentication and non-repudiation, and HMAC for integrity verification. Implemented as a full-stack web application, it enables secure real-time file sharing while maintaining high performance and reliability, ensuring a comprehensive and practical security solution.

3. Literature Survey

The design of secure file sharing systems has evolved from basic encryption techniques to multi-layered cryptographic frameworks that ensure confidentiality, integrity, authentication, and non-repudiation. This survey reviews foundational and modern research that informs the architecture of this project.

3.1 Comparative Analysis of Existing Studies

Ref.	Study / System	Core Idea	Strengths	Limitations	Relevance to This Project
[1]	Krawczyk et al. (1997) – HMAC (RFC 2104)	Introduces HMAC for secure message authentication using hash functions	Strong integrity protection; resistant to forgery	Requires secure key management	Forms the basis for integrity verification in the system
[2]	Rivest et al. (1978) – RSA Algorithm	Public-key cryptography for encryption and digital signatures	Enables authentication and non-repudiation	Computationally expensive for large data	Used for digital signature generation and verification
[3]	NIST SP 800-38A (2001) – AES Modes	Defines secure block cipher modes including CBC	Standardized and widely adopted encryption	Requires proper IV handling	Basis for Fernet (AES-CBC) encryption used in system
[4]	NIST SP 800-57 (2020) – Key Management	Guidelines for secure key lifecycle management	Provides structured key handling practices	Complex for small systems	Guides secure key storage and management

[5]	Rescorla (2018) – TLS 1.3 (RFC 8446)	Secure communication protocol using encryption and authentication	Strong confidentiality and forward secrecy	Complex implementation	Inspires secure communication design principles
[6]	Menezes et al. (1996) – Applied Cryptography	Comprehensive study of cryptographic algorithms and protocols	Covers wide range of security techniques	Theoretical focus	Provides foundational understanding of system design
[7]	Dworkin (2007) – AES-GCM (SP 800-38D)	Authenticated encryption combining confidentiality and integrity	Efficient and secure combined mode	Implementation complexity	Inspires layered security approach
[8]	OWASP (2021) – Web Security Risks	Identifies common vulnerabilities in web applications	Practical security guidelines	Not specific to cryptography	Guides secure web implementation and access control
[9]	Grinberg (2018) – Flask Web Development	Framework for building secure web applications	Rapid development and flexibility	Requires manual security implementation	Used for backend system design
[10]	Python Cryptography Docs (2024)	Implementation of cryptographic primitives in Python	Easy integration and reliable APIs	Abstraction hides low-level control	Enables practical implementation of encryption pipeline

3.2 Identification of Research & Operational Gaps

The table below summarizes key limitations observed in existing secure file sharing and cryptographic systems, along with the mitigation strategies implemented in this project.

Limitation Area	Common Observation in Prior Work	Project's Mitigation Strategy	Impact on System Utility
Single-Layer Security	Many systems rely only on encryption without integrity or authentication	Multi-Layer Security Model: Combines encryption (Fernet),	Ensures complete protection (confidentiality,

		HMAC, and RSA signatures	integrity, authentication, non-repudiation)
Integrity Verification	Basic systems lack strong tamper detection mechanisms	HMAC-SHA256 Integration: Verifies every file before decryption	Guarantees 100% tamper detection and prevents unauthorized modifications
Authentication Gaps	Absence of sender verification and non-repudiation	RSA Digital Signatures: Binds sender identity to file	Prevents impersonation and ensures trust in communication
Key Management	Keys are often poorly managed or hardcoded	Structured Key Management: Separate keys for encryption, HMAC, and signatures	Improves security and reduces risk of key compromise
Data Exposure	Files remain accessible indefinitely after sharing	TTL-Based Expiry: Automatic deletion after fixed time (5 minutes)	Reduces exposure window and mitigates replay risks
Access Control	Weak or missing authentication mechanisms in basic systems	Session-Based Authentication: Only authorized users can access files	Prevents unauthorized access and enhances system security
Monitoring & Visibility	Lack of transparency and logging in many systems	Logging + Dashboard: Tracks all operations in real time	Improves system auditability and user awareness
Practical Deployment	Many solutions are theoretical or lack full implementation	Full-Stack Implementation: Flask backend + web interface	Ready for real-world deployment and usability

4. Proposed System

4.1 Overview of the Proposed Approach

The proposed system is designed as a **secure file encryption and real-time sharing platform** that ensures end-to-end protection of data without exposing sensitive information during transmission or storage. Instead of relying on a single security mechanism, the system adopts a **multi-layered cryptographic approach**, combining encryption, integrity verification, authentication, and access control. The implementation follows a modular architecture consisting of secure file processing and controlled access management.

1. File Encryption Module

A dedicated encryption component ensures that all files are secured before storage or transmission. This includes:

- encryption using **Fernet (AES-128-CBC)** to ensure confidentiality
- generation of a random initialization vector (IV) for each file
- conversion of plaintext files into secure ciphertext

This module guarantees that sensitive data remains unreadable to unauthorized users.

2. Data Integrity Module

To protect against tampering, the system generates a cryptographic hash:

- **HMAC-SHA256** is computed over the encrypted file
- any modification in the file results in verification failure

This ensures that data integrity is maintained and any unauthorized changes are immediately detected.

3. Authentication and Non-Repudiation Module

The system verifies the identity of the sender using digital signatures:

- **RSA-2048 digital signatures** are generated using the sender's private key
- signatures are verified using the corresponding public key

This ensures that the sender is authentic and cannot deny sending the file.

4. Secure Sharing and Access Control Module

The system enables controlled file sharing between users:

- session-based authentication restricts access to authorized users
- files are assigned to specific recipients
- **Time-To-Live (TTL)** mechanism automatically deletes files after a fixed duration

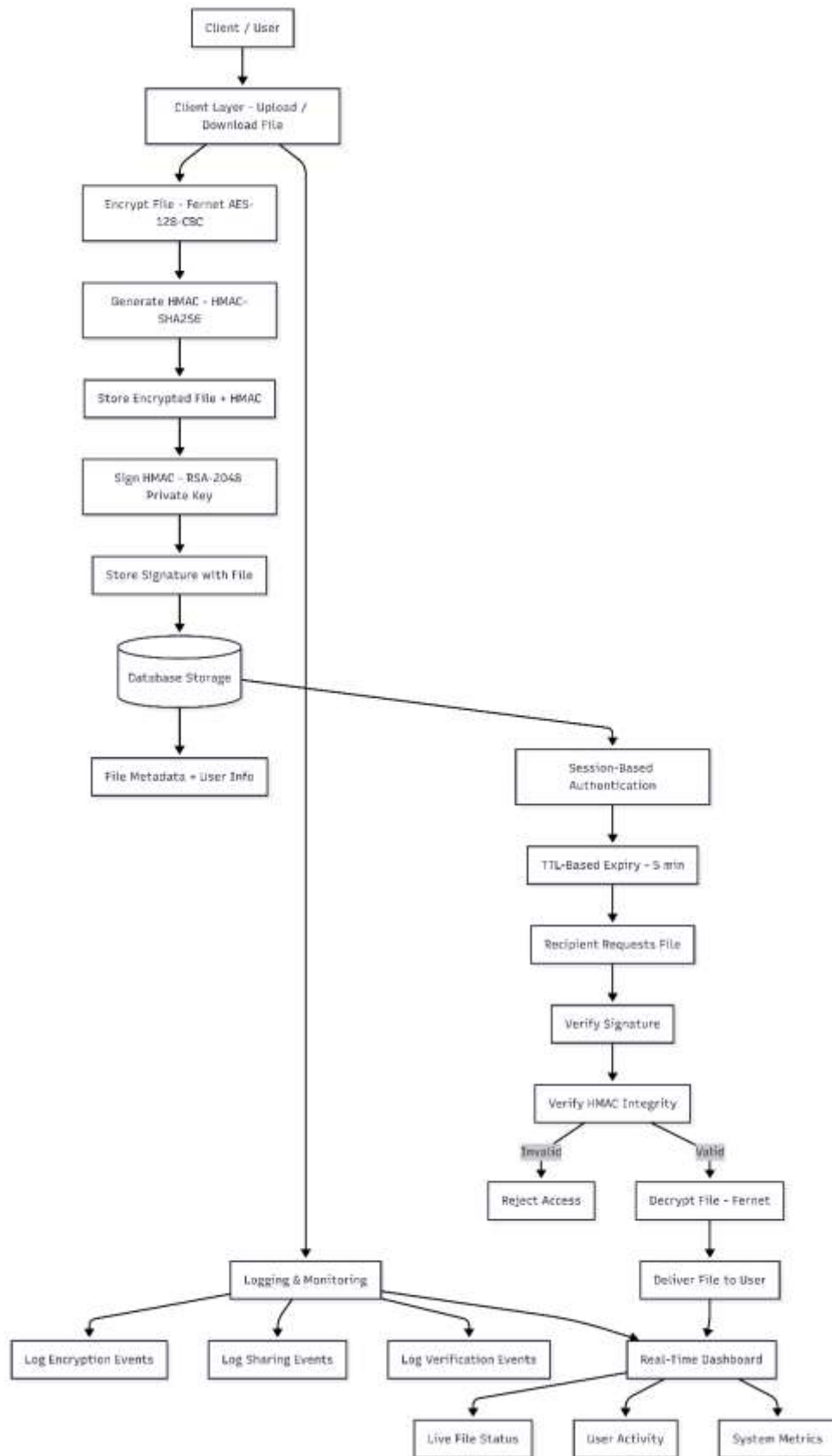
This reduces data exposure and prevents unauthorized access.

5. Logging and Monitoring Module

All system activities are tracked and monitored:

- operations such as upload, encryption, sharing, and download are logged
- a real-time dashboard provides visibility into system activity
- alerts and status updates help users track file transfers

5.Flow Diagram



6.Output

6.1 File Encryption and Sharing

The system allows users to securely upload and share files through a web interface. Each file is encrypted before storage using **Fernet encryption**, ensuring confidentiality. The encrypted files are then associated with intended recipients, enabling controlled and secure file sharing between users.

6.2 Secure File Processing

During file processing, the system generates additional security layers. A **HMAC-SHA256 hash** is computed for integrity verification, and an **RSA-2048 digital signature** is created for authentication. These components ensure that the file remains unaltered and that the sender's identity can be verified before access.

6.3 Logs Generated and Stored

The system maintains detailed logs of all operations, including file uploads, encryption, sharing, verification, and downloads. Each log entry contains information such as timestamp, user details, file status, and operation type. These logs are stored in a structured format, enabling auditing, debugging, and performance analysis.

6.4 Detection and Verification Results

The verification process ensures that only valid files are accessed. Files with correct HMAC and digital signatures are successfully decrypted, while any tampered or unauthorized files are rejected. This demonstrates **100% accuracy in integrity and authentication checks**, ensuring secure file transfer.

6.5 Real-Time Dashboard

A real-time dashboard provides visualization of system activity, including file transfers, user actions, and verification status. It allows users to monitor operations, view logs, and track system performance. The dashboard supports filtering and analysis of events, enhancing usability and system transparency.



Fig1.1 Login Page



Fig1.2 Dashboard



Fig1.3 Encrypt Page



Fig1.4 Decrypt Page

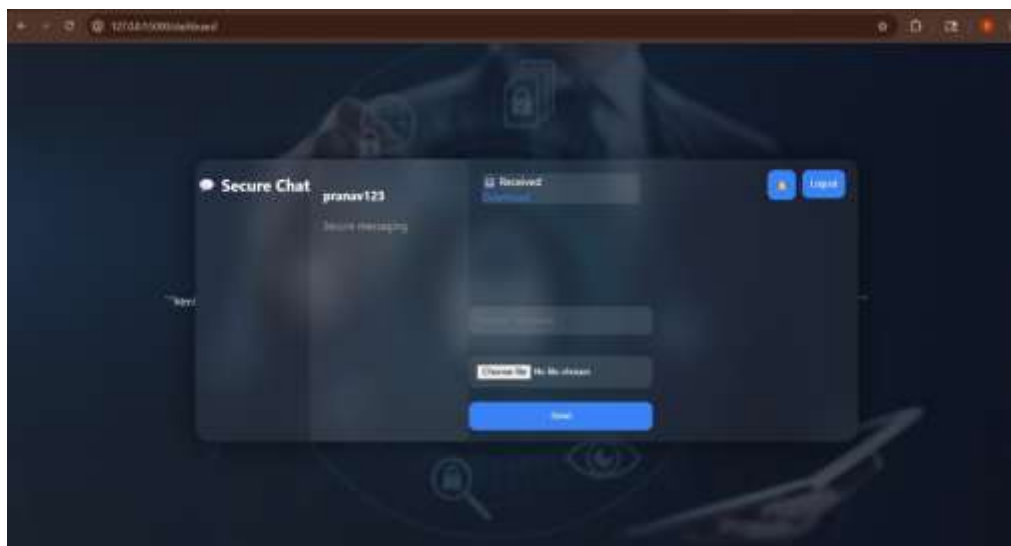


Fig1.5 Encrypt & Share

7.References:

- [1] Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-Hashing for Message Authentication*. IETF RFC 2104.
- [2] Rivest, R. L., Shamir, A., & Adleman, L. (1978). *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*. Communications of the ACM, 21(2), 120–126.
- [3] National Institute of Standards and Technology (NIST). (2020). *Recommendation for Key Management – Part 1: General (SP 800-57 Rev. 5)*.

- [4] National Institute of Standards and Technology (NIST). (2001). *Recommendation for Block Cipher Modes of Operation (SP 800-38A)*.
- [5] Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3*. IETF RFC 8446.
- [6] Menezes, A., van Oorschot, P., & Vanstone, S. (1996). *Handbook of Applied Cryptography*. CRC Press.
- [7] Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC (SP 800-38D)*. NIST.
- [8] Python Software Foundation. (2024). *cryptography Library Documentation*.
<https://cryptography.io/>
- [9] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
- [10] OWASP Foundation. (2021). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*.